

动态规划算法入门

程序设计竞赛集训营

太原理工大学

2026-02-24

Outline

1. 编程的本质?	2	3.2.3 最大公共子序列问题 .	36
1.1 人类的本质是复读机	3	3.2.4 路径数问题	40
1.2 程序中的递归	6	3.2.5 魔法石	43
1.3 递归的展开	9	4. 总结	46
1.4 尾递归优化	12	5. 扩展: 现代语言	48
1.5 案例: 幂次算法	21	5.1 回看硬币找零	50
2. 记忆化递归	26	5.2 魔法石问题偏函数式解法 .	51
2.1 问题过于复杂?	27	5.3 惰性求值硬币问题 (选) .	52
2.2 记忆化方法	28		
3. 动态规划	30		
3.1 什么是动态规划?	31		
3.2 典型的动态规划问题	32		
3.2.1 硬币找零问题	32		
3.2.2 硬币问题变体	35		

1. 编程的本质？

每一种强有力的语言，都为此（能够将简单的认识组合起来形成更复杂认识的方法）提供了三种机制：

- **基本表达形式**：用于表示语言所关心的最简单的个体。
- **组合的方法**：通过它们可以从较简单的东西出发构造出复合的元素。
- **抽象的方法**：通过它们可以为复合对象命名，并将它们当作单元去操作。

——《计算机程序的构造和解释》

1.1 人类的本质是复读机

1. 编程的本质？

人类的大脑，是一个内存极小的电脑，不擅长记忆大量的临时变量。

人类的大脑，是一个内存极小的电脑，不擅长记忆大量的临时变量。
我们更加擅长思考如何将复杂问题，拆解成更小的子问题。

人类的大脑，是一个内存极小的电脑，不擅长记忆大量的临时变量。

我们更加擅长思考如何将复杂问题，拆解成更小的子问题。

两个典型的例子：

$$n! = \begin{cases} 1, & n = 0 \\ n \cdot (n-1)!, & n > 0 \end{cases}$$

$$F(n) = \begin{cases} n, & n \leq 1 \\ F(n-1) + F(n-2), & n > 1 \end{cases}$$

它们有一个共同点：都是通过将复杂问题分解为更小的子问题来解决的。

它们有一个共同点：都是通过将复杂问题分解为更小的子问题来解决的。

假如我们已经知道了如何把一个复杂问题变成更加简单的问题，

它们有一个共同点：都是通过将复杂问题分解为更小的子问题来解决的。

假如我们已经知道了如何把一个复杂问题变成更加简单的问题，而最简单的问题我们又已经知道答案了。

它们有一个共同点：都是通过将复杂问题分解为更小的子问题来解决的。

假如我们已经知道了如何把一个复杂问题变成更加简单的问题，而最简单的问题我们又已经知道答案了。

那就意味着，所有的问题我们都能解决了！

(数学归纳法的思路)

1.2 程序中的递归

那么我们该如何在程序中实现这种“分解问题”的思路呢？

1.2 程序中的递归

那么我们该如何在程序中实现这种“分解问题”的思路呢？

答案是：递归。

1.2 程序中的递归

那么我们该如何在程序中实现这种“分解问题”的思路呢？

答案是：递归。

以上代码几乎可以直接翻译成递归函数：

cpp

```
1 unsigned long long factorial(unsigned int n) {
```

```
2     if (n == 0) return 1;
```

> 一般而言，递归函数包含两个部分：基准情形和递推情形。

```
3     return n * factorial(n - 1);
```

> 注意，这里没有放入到 else 当中，因为前面相当于是一个守卫，不满足条件的情况已经返回了，这里就可以少一层缩进。

```
4 }
```

```
1 unsigned long long factorial(unsigned int n) {  
2     if (n == 0) return 1;  
3     return n * factorial(n - 1);  
4 }
```

为何函数在还没有完成定义的时候就可以调用自己呢？

答案是：因为函数定义的时候，函数体内的代码**并不会立刻执行**。

当函数被调用时，会创建一个新的执行环境，这个环境中包含了函数的参数和局部变量。

1.2 程序中的递归

这个函数的定义，相当于是把系列的指令打包起来，放在一个盒子里，并将其关联到一个名字上。

当我们调用这个函数时，会先传入参数的值，然后执行打包好的指令。

当执行到自身的调用时，会更新传入的参数，并在全局环境中查找函数的定义，然后在新的执行环境中执行函数体内的代码。

1.3 递归的展开

让我们来看看递归函数在运行时的具体过程：

$$n! = \begin{cases} 1, & n = 0 \\ n \cdot (n - 1)!, & n > 0 \end{cases}$$

5!

1.3 递归的展开

让我们来看看递归函数在运行时的具体过程：

$$n! = \begin{cases} 1, & n = 0 \\ n \cdot (n - 1)!, & n > 0 \end{cases}$$

5!

= 5 × 4!

1.3 递归的展开

让我们来看看递归函数在运行时的具体过程：

$$n! = \begin{cases} 1, & n = 0 \\ n \cdot (n - 1)!, & n > 0 \end{cases}$$

5!

$$= 5 \times 4! = 5 \times (4 \times 3!)$$

1.3 递归的展开

让我们来看看递归函数在运行时的具体过程：

$$n! = \begin{cases} 1, & n = 0 \\ n \cdot (n-1)!, & n > 0 \end{cases}$$

5!

$$= 5 \times 4! = 5 \times (4 \times 3!) = 5 \times (4 \times (3 \times 2!))$$

1.3 递归的展开

让我们来看看递归函数在运行时的具体过程：

$$n! = \begin{cases} 1, & n = 0 \\ n \cdot (n - 1)!, & n > 0 \end{cases}$$

5!

$$= 5 \times 4! = 5 \times (4 \times 3!) = 5 \times (4 \times (3 \times 2!)) = 5 \times (4 \times (3 \times (2 \times 1!)))$$

1.3 递归的展开

让我们来看看递归函数在运行时的具体过程：

$$n! = \begin{cases} 1, & n = 0 \\ n \cdot (n - 1)!, & n > 0 \end{cases}$$

5!

$$= 5 \times 4! = 5 \times (4 \times 3!) = 5 \times (4 \times (3 \times 2!)) = 5 \times (4 \times (3 \times (2 \times 1!)))$$

$$= 5 \times (4 \times (3 \times (2 \times 1)))$$

1.3 递归的展开

让我们来看看递归函数在运行时的具体过程：

$$n! = \begin{cases} 1, & n = 0 \\ n \cdot (n-1)!, & n > 0 \end{cases}$$

5!

$$= 5 \times 4! = 5 \times (4 \times 3!) = 5 \times (4 \times (3 \times 2!)) = 5 \times (4 \times (3 \times (2 \times 1!)))$$

$$= 5 \times (4 \times (3 \times (2 \times 1)))$$

$$= 5 \times (4 \times (3 \times 2))$$

1.3 递归的展开

让我们来看看递归函数在运行时的具体过程：

$$n! = \begin{cases} 1, & n = 0 \\ n \cdot (n-1)!, & n > 0 \end{cases}$$

5!

$$= 5 \times 4! = 5 \times (4 \times 3!) = 5 \times (4 \times (3 \times 2!)) = 5 \times (4 \times (3 \times (2 \times 1!)))$$

$$= 5 \times (4 \times (3 \times (2 \times 1)))$$

$$= 5 \times (4 \times (3 \times 2))$$

$$= 5 \times (4 \times 6)$$

1.3 递归的展开

让我们来看看递归函数在运行时的具体过程：

$$n! = \begin{cases} 1, & n = 0 \\ n \cdot (n-1)!, & n > 0 \end{cases}$$

5!

$$= 5 \times 4! = 5 \times (4 \times 3!) = 5 \times (4 \times (3 \times 2!)) = 5 \times (4 \times (3 \times (2 \times 1!)))$$

$$= 5 \times (4 \times (3 \times (2 \times 1)))$$

$$= 5 \times (4 \times (3 \times 2))$$

$$= 5 \times (4 \times 6)$$

$$= 5 \times 24$$

1.3 递归的展开

让我们来看看递归函数在运行时的具体过程：

$$n! = \begin{cases} 1, & n = 0 \\ n \cdot (n-1)!, & n > 0 \end{cases}$$

5!

$$= 5 \times 4! = 5 \times (4 \times 3!) = 5 \times (4 \times (3 \times 2!)) = 5 \times (4 \times (3 \times (2 \times 1!)))$$

$$= 5 \times (4 \times (3 \times (2 \times 1)))$$

$$= 5 \times (4 \times (3 \times 2))$$

$$= 5 \times (4 \times 6)$$

$$= 5 \times 24$$

$$= 120$$

1.3 递归的展开

我们再来看一下斐波那契数列的递归展开：

$$F(4)$$

1.3 递归的展开

我们再来看一下斐波那契数列的递归展开：

$$F(4) = F(3) + F(2)$$

1.3 递归的展开

我们再来看一下斐波那契数列的递归展开：

$$\begin{aligned} F(4) &= F(3) + F(2) \\ &= (F(2) + F(1)) + F(2) \end{aligned}$$

1.3 递归的展开

我们再来看一下斐波那契数列的递归展开：

$$\begin{aligned} F(4) &= F(3) + F(2) \\ &= (F(2) + F(1)) + F(2) \\ &= ((F(1) + F(0)) + F(1)) + F(2) \end{aligned}$$

1.3 递归的展开

我们再来看一下斐波那契数列的递归展开：

$$\begin{aligned} F(4) &= F(3) + F(2) \\ &= (F(2) + F(1)) + F(2) \\ &= ((F(1) + F(0)) + F(1)) + F(2) \\ &= ((1 + 0) + 1) + F(2) \end{aligned}$$

1.3 递归的展开

我们再来看一下斐波那契数列的递归展开：

$$\begin{aligned} F(4) &= F(3) + F(2) \\ &= (F(2) + F(1)) + F(2) \\ &= ((F(1) + F(0)) + F(1)) + F(2) \\ &= ((1 + 0) + 1) + F(2) \\ &= 2 + F(2) \end{aligned}$$

1.3 递归的展开

我们再来看一下斐波那契数列的递归展开：

$$\begin{aligned} F(4) &= F(3) + F(2) \\ &= (F(2) + F(1)) + F(2) \\ &= ((F(1) + F(0)) + F(1)) + F(2) \\ &= ((1 + 0) + 1) + F(2) \\ &= 2 + F(2) \\ &= 2 + (F(1) + F(0)) \end{aligned}$$

1.3 递归的展开

我们再来看一下斐波那契数列的递归展开：

$$\begin{aligned} F(4) &= F(3) + F(2) \\ &= (F(2) + F(1)) + F(2) \\ &= ((F(1) + F(0)) + F(1)) + F(2) \\ &= ((1 + 0) + 1) + F(2) \\ &= 2 + F(2) \\ &= 2 + (F(1) + F(0)) \\ &= 2 + (1 + 0) \end{aligned}$$

1.3 递归的展开

我们再来看一下斐波那契数列的递归展开：

$$\begin{aligned} F(4) &= F(3) + F(2) \\ &= (F(2) + F(1)) + F(2) \\ &= ((F(1) + F(0)) + F(1)) + F(2) \\ &= ((1 + 0) + 1) + F(2) \\ &= 2 + F(2) \\ &= 2 + (F(1) + F(0)) \\ &= 2 + (1 + 0) = 3 \end{aligned}$$

1.3 递归的展开

可以看到，这个展开是二维的。可以通过计算得到， $F(n)$ 总共需要计算 $2F(n-1) + 1$ 次。也就是大概是 $O(2^n)$ 的时间复杂度。

1.3 递归的展开

可以看到，这个展开是二维的。可以通过计算得到， $F(n)$ 总共需要计算 $2F(n-1) + 1$ 次。也就是大概是 $O(2^n)$ 的时间复杂度。

我们可以实验一下这个计算方法在 C++ 语言中的计算速度：

```
1 unsigned long long fibonacci(unsigned int n) {  
2     if (n <= 1) return n;  
3     return fibonacci(n - 1) + fibonacci(n - 2);  
4 }
```

当 $n = 50$ 时，已经基本无法在合理时间内计算出结果了。

1.4 尾递归优化

可以看到，我们需要不断地得到临时的计算结果，也就是所谓的**中间状态**，有时也被称为**栈帧**。对于阶乘而言，是一个一维的展开。

1.4 尾递归优化

可以看到，我们需要不断地得到临时的计算结果，也就是所谓的**中间状态**，有时也被称为**栈帧**。对于阶乘而言，是一个一维的展开。

这种写法实际上是非常低效的，因为它会重复计算很多中间结果。那么有什么办法可以避免这种重复计算呢？

1.4 尾递归优化

可以看到，我们需要不断地得到临时的计算结果，也就是所谓的**中间状态**，有时也被称为**栈帧**。对于阶乘而言，是一个一维的展开。

这种写法实际上是非常低效的，因为它会重复计算很多中间结果。那么有什么办法可以避免这种重复计算呢？

一种简单有效的的方法是，我们把中间结果手动保存到参数里面。

1.4 尾递归优化

1. 编程的本质？

```
1 unsigned long long factorial_helper(  
2     unsigned int n,  
3     unsigned long long accumulator) {  
4     if (n == 0) return accumulator;  
5     return factorial_helper(n - 1, n * accumulator);  
6 }  
7  
8 unsigned long long factorial(unsigned int n) {  
9     return factorial_helper(n, 1);  
10 }
```

1.4 尾递归优化

或者，利用参数默认值简化写法：

```
1 unsigned long long factorial(  
2     unsigned int n,  
3     unsigned long long accumulator = 1) {  
4     if (n == 0) return accumulator;  
5     return factorial(n - 1, n * accumulator);  
6 }
```

1.4 尾递归优化

现在，函数的展开方式为：

5!

1.4 尾递归优化

现在，函数的展开方式为：

$$5! = f(5, 1)$$

1.4 尾递归优化

现在，函数的展开方式为：

$$\begin{aligned} 5! &= f(5, 1) \\ &= f(4, 5 \times 1 = 5) \end{aligned}$$

1.4 尾递归优化

现在，函数的展开方式为：

$$\begin{aligned} 5! &= f(5, 1) \\ &= f(4, 5 \times 1 = 5) \\ &= f(3, 4 \times 5 = 20) \end{aligned}$$

1.4 尾递归优化

1. 编程的本质？

现在，函数的展开方式为：

$$\begin{aligned} 5! &= f(5, 1) \\ &= f(4, 5 \times 1 = 5) \\ &= f(3, 4 \times 5 = 20) \\ &= f(2, 3 \times 20 = 60) \end{aligned}$$

1.4 尾递归优化

1. 编程的本质？

现在，函数的展开方式为：

$$\begin{aligned} 5! &= f(5, 1) \\ &= f(4, 5 \times 1 = 5) \\ &= f(3, 4 \times 5 = 20) \\ &= f(2, 3 \times 20 = 60) \\ &= f(1, 2 \times 60 = 120) \end{aligned}$$

1.4 尾递归优化

现在，函数的展开方式为：

$$\begin{aligned} 5! &= f(5, 1) \\ &= f(4, 5 \times 1 = 5) \\ &= f(3, 4 \times 5 = 20) \\ &= f(2, 3 \times 20 = 60) \\ &= f(1, 2 \times 60 = 120) \\ &= f(0, 120) \end{aligned}$$

1.4 尾递归优化

现在，函数的展开方式为：

$$\begin{aligned} 5! &= f(5, 1) \\ &= f(4, 5 \times 1 = 5) \\ &= f(3, 4 \times 5 = 20) \\ &= f(2, 3 \times 20 = 60) \\ &= f(1, 2 \times 60 = 120) \\ &= f(0, 120) = 120 \end{aligned}$$

1.4 尾递归优化

1. 编程的本质？

需要注意的是，这一特性需要编程语言是严格求值的顺序：即先对参数的值进行计算，然后再调用函数。不过幸运的是，大多数主流编程语言都是如此的。

1.4 尾递归优化

需要注意的是，这一特性需要编程语言是严格求值的顺序：即先对参数的值进行计算，然后再调用函数。不过幸运的是，大多数主流编程语言都是如此的。

我们再来看看斐波那契数列的尾递归写法：

```
1 unsigned long long fibonacci(  
2     unsigned int n,  
3     unsigned long long a = 0,  
4     unsigned long long b = 1) {  
5     if (n == 0) return a;  
6     return fibonacci(n - 1, b, a + b);  
7 }
```

1.4 尾递归优化

当我们函数最终只是一个自身的重复，我们要做的只是更新参数的值时，编译器就可以进行**尾递归优化**，将递归调用转换为循环，从而避免了大量的中间状态保存。

1.4 尾递归优化

当我们函数最终只是一个自身的重复，我们要做的只是更新参数的值时，编译器就可以进行**尾递归优化**，将递归调用转换为循环，从而避免了大量的中间状态保存。

这是因为，尾递归实际上非常好优化，我们可以手动来实现以上递归函数的循环版本：

```
1 unsigned long long factorial(unsigned int n) {
2     unsigned long long accumulator = 1;
3     while (n > 0) {
4         // Update two parameters
5         accumulator *= n;
6         n--;
7     }
8     return accumulator;
9 }
```

1.4 尾递归优化

1. 编程的本质？

```
1 unsigned long long fibonacci(unsigned int n) {
2     unsigned long long a = 0;
3     unsigned long long b = 1;
4     while (n > 0) {
5         // Update three parameters
6         unsigned long long temp = a;
7         a = b;
8         b = temp + b;
9         n--;
10    }
11    return a;
12 }
```


1.4 尾递归优化

所以，任何循环，都可以被重写为尾递归函数。

递归函数的优势在于：

- 直观展示了问题的分解过程，更加符合人类的思维习惯。
- 可以写出无变量的代码，减少心智负担。
- 如果优化为尾递归，可以被编译器优化为循环，并无性能损失。
- 基于数学归纳法的思路，可以在很大程度上保证代码的正确性。

1.4 尾递归优化

所以，任何循环，都可以被重写为尾递归函数。

递归函数的优势在于：

- 直观展示了问题的分解过程，更加符合人类的思维习惯。
- 可以写出无变量的代码，减少心智负担。
- 如果优化为尾递归，可以被编译器优化为循环，并无性能损失。
- 基于数学归纳法的思路，可以在很大程度上保证代码的正确性。

不过也存在一些小问题：

- 中间状态过多时，需要维护大量的参数，代码可读性下降。
- 并非所有语言都支持尾递归优化，可能存在栈溢出风险。
- 不太方便调试。

1.5 案例：幂次算法

一个数的 n 次幂，可以定义为：

$$x^n = \begin{cases} 1, & n = 0 \\ x \cdot x^{n-1}, & n > 0 \end{cases}$$

可以写为（挑战一下看看能否尾递归优化）：

```
1 using Result = long long;
2 using Input = int;
3
4 Result power(Input x, Input n) {
5     return n == 0 ? 1 : x * power(x, n - 1);
6 }
```

1.5 案例：幂次算法

1. 编程的本质？

但是，我们还可以用以下思路进行优化：

如果 n 是偶数，那么 $x^n = x^{\frac{n}{2}} \cdot x^{\frac{n}{2}}$

如果 n 是奇数，那么 $x^n = x \cdot x^{\frac{n-1}{2}} \cdot x^{\frac{n-1}{2}}$

1.5 案例：幂次算法

可以写为：

```
1 using Result = long long;
2 using Input = int;
3
4 Result quick_power(Input x, Input n) {
5     if (n == 0) {
6         return 1;
7     }
8     Result r = quick_power(x, n / 2);
9     return n % 2 == 0 ? r * r : x * r * r;
10 }
```

1.5 案例：幂次算法

那么我们应该如何进行尾递归优化？

1.5 案例：幂次算法

1. 编程的本质？

那么我们应该如何进行尾递归优化？

首先重新认识这一过程：我们可以认为， x^n ，变换成了 $(x^2)^{\frac{n}{2}}$ ，再往后变换。

1.5 案例：幂次算法

1. 编程的本质？

那么我们应该如何进行尾递归优化？

首先重新认识这一过程：我们可以认为， x^n ，变换成了 $(x^2)^{\frac{n}{2}}$ ，再往后变换。

同时，还需要一个记录在变换过程中，有哪些值需要额外乘一下：

$$x^n = x \cdot \left((x^2)^{\frac{n-1}{2}} \right), \text{ 此时多乘了一个 } x。$$

1.5 案例：幂次算法

那么我们应该如何进行尾递归优化？

首先重新认识这一过程：我们可以认为， x^n ，变换成了 $(x^2)^{\frac{n}{2}}$ ，再往后变换。

同时，还需要一个记录在变换过程中，有哪些值需要额外乘一下：

$$x^n = x \cdot \left((x^2)^{\frac{n-1}{2}} \right), \text{ 此时多乘了一个 } x。$$

最后，会得到一个 x^1 的形式，再变换，得到 $x \cdot (x^2)^0 = x \cdot 1 = x$ 。

于是，最终可以写成：

1.5 案例：幂次算法

1. 编程的本质？

```
1 using Result = long long;
2 using Input = int;
3
4 Result better_quick_power(Input x, Input n, Input acc = 1) {
5     if (n == 0)
6         return acc;
7     if (n == 1)
8         return x * acc;
9     if (n % 2 == 0)
10        return better_quick_power(x * x, n / 2, acc);
11    return better_quick_power(x * x, n / 2, acc * x);
12 }
```

2. 记忆化递归

2.1 问题过于复杂？

- 并不是所有的问题都能被简单地进行尾递归优化。

- 并不是所有的问题都能被简单地进行尾递归优化。
- 也就是说，虽然所有的循环都可以被写成递归形式，但是并非所有的递归都可以写成循环形式。

- 并不是所有的问题都能被简单地进行尾递归优化。
- 也就是说，虽然所有的循环都可以被写成递归形式，但是并非所有的递归都可以写成循环形式。
- 还有一点，有时把递归进行尾递归优化，往往会带来很大的心智负担。

我们再来看一下斐波那契数列

如果我们觉得 $F(n)$ 本质上是两个数字反复叠加这一规律难以发现, 同时又觉得递归形式过于复杂, 有什么办法能够既保留最简单的问题分解描述形式, 又保证足够的性能呢?

我们可以采用一种策略: **结果记忆**, 又被称为 **memoize**。

有很多按照函数式编程的方式实现的 memoize 的库¹, 不过, 我们这里只采用手动的方式来实现最简单的记忆化策略。

¹<https://github.com/jimporter/memo>

```
1 #include <iostream>
2 #include <unordered_map>
3
4 using Result = unsigned long long;
5 using Input = unsigned int;
6 using Record = std::unordered_map<Input, Result>;
7
8 Result memo_fib(Input n, Record &rec) {
9     if (n < 2) {
10         return n;
11     }
12     if (rec.find(n) != rec.end()) {
13         return rec[n];
14     }
15     Result r = memo_fib(n - 1, rec) + memo_fib(n - 2, rec);
16     rec[n] = r;
17     return r;
18 }
19
20 int main() {
21     Record rec;
22     std::cout << memo_fib(50, rec);
23 }
```


3. 动态规划

3.1 什么是动态规划？

其实，我们之前的思路，已经把动态规划的全部内容说完了：

3.1 什么是动态规划？

其实，我们之前的思路，已经把动态规划的全部内容说完了：

- 明确如何将复杂问题分解为更加简单的问题

3.1 什么是动态规划？

其实，我们之前的思路，已经把动态规划的全部内容说完了：

- 明确如何将复杂问题分解为更加简单的问题
- 明确最简单问题的解

3.1 什么是动态规划？

其实，我们之前的思路，已经把动态规划的全部内容说完了：

- 明确如何将复杂问题分解为更加简单的问题
- 明确最简单问题的解
- 记忆中忆计算的结果

3.1 什么是动态规划？

其实，我们之前的思路，已经把动态规划的全部内容说完了：

- 明确如何将复杂问题分解为更加简单的问题
- 明确最简单问题的解
- 记忆中忆计算的结果

这就是全部了。不过大家看到真正的竞赛类解法时，很少用 `unordered_map` 或者 `map` 来记忆结果，而是采用在栈上分配数组来实现记忆化。这样做的好处是性能高，坏处是代码可读性差，心智负担略高。本课程面向想要了解竞赛的同学，因此采用更加简单的 `map` 来实现记忆化。

3.2 典型的动态规划问题

3.2.1 硬币找零问题

以下问题选自欧拉计划第 31 题¹，是一个非常经典的动态规划问题。

英国的货币单位分为英镑 (£) 和便士 (p)。目前流通的硬币一共有八种面值：

$$1p, 2p, 5p, 10p, 20p, 50p, \text{£}1(100p), \text{£}2(200p)$$

想要凑出 £2，其中一种做法是：

$$1 \times \text{£}1 + 1 \times 50p + 2 \times 20p + 1 \times 5p + 1 \times 2p + 3 \times 1p$$

不限制硬币数量，凑出 £2 有多少种不同的做法？

¹<https://projecteuler.net/problem=31>

3.2 典型的动态规划问题

我们的思路是：

对于要凑的钱 n ，以及现有的硬币： $x = \{x_1, x_2, \dots, x_c\}$ ，其中， c 是硬币的数量，我们有以下分解思路：

3.2 典型的动态规划问题

我们的思路是：

对于要凑的钱 n ，以及现有的硬币： $x = \{x_1, x_2, \dots, x_c\}$ ，其中， c 是硬币的数量，我们有以下分解思路：

分两种情况，

1. 我们先尝试花一下 x_c ，那么问题变为了 $n - x_c$ 和 x
2. 我们不使用 x_c ，那么问题变成了 n 和 $x \setminus \{x_c\}$
3. 如果现在的目标变成了0，说明找零成功了，返回1
4. 如果目标变成了负数，或者目标大于零的前提下，没有硬币了，则找零失败，返回0

3.2 典型的动态规划问题

3. 动态规划

```
1 #include <iostream>
2 #include <map>
3 #include <tuple>
4 #include <vector>
5
6 using Money = int; // 需要负数, 所有不是unsigned int
7 using Coins = std::vector<Money>;
8 using CoinsNum = size_t;
9 using Input = std::tuple<Money, CoinsNum>;
10 using Result = unsigned long long;
11 // tuple 作为 unordered_map 的 key 需要实现 hash
12 // 与本课程无关, 因此直接采用 map, 性能会略有损失
13 using Record = std::map<Input, Result>;
14
15 Result solve(Money target,
16              const Coins &coins,
17              CoinsNum n,
18              Record &rec) {
19     if (target == 0)
20         return 1;
21     if (target < 0 || n <= 0)
22         return 0;
23
24     Input input = std::make_tuple(target, n);
25     if (rec.find(input) != rec.end()) {
```

```
26         return rec[input];
27     }
28
29     Money current = coins[n - 1];
30     Result r =
31         solve(target - current, coins, n, rec) +
32         solve(target, coins, n - 1, rec);
33     rec[input] = r;
34     return r;
35 }
36
37 int main() {
38     Coins coins = {1, 2, 5, 10, 20, 50, 100, 200};
39     Record rec;
40     std::cout << solve(200, coins, coins.size(), rec);
41 }
```

3.2 典型的动态规划问题

3.2.2 硬币问题变体

有一种硬币找零问题的变体：给定硬币种类和目标钱数，问凑齐目标钱数最少需要多少个硬币？-1表示无法凑出目标钱数。

这个问题作为挑战大家自行完成。

3.2 典型的动态规划问题

3.2.3 最大公共子序列问题

此问题选自 LeetCode 1143:

给定两个字符串 `text1` 和 `text2`，返回这两个字符串的最长**公共子序列**的长度。如果不存在公共子序列，返回 0。

一个字符串的**子序列**是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。

例如，“ace” 是 “abcde” 的子序列，但 “aec” 不是 “abcde” 的子序列。

两个字符串的**公共子序列**是这两个字符串所共同拥有的子序列。

3.2 典型的动态规划问题

问题分解：

对于字符串 s_1 和 s_2 ：

1. $f(i, j)$ 表示 s_1 的前 i 个字符和 s_2 的前 j 个字符的最长公共子序列
2. 如果 $s_1[i] = s_2[j]$, 则 $f(i, j) = f(i - 1, j - 1) + 1$
3. 否则, $f(i, j) = \max\{f(i - 1, j), f(i, j - 1)\}$
4. i 或者 j 为 0 时, $f(i, j) = 0$

3.2 典型的动态规划问题

3. 动态规划

```
1 #include <iostream>
2 #include <map>
3 #include <string>
4 #include <tuple>
5
6 using Index = int;
7 using Input = std::tuple<Index, Index>;
8 using Output = Index;
9 using Record = std::map<Input, Output>;
10
11 Output lcm_helper(const std::string &s1,
12                  const std::string &s2,
13                  Index i, Index j,
14                  Record &rcd) {
15     if (i < 0 || j < 0)
16         return 0;
17     Input input = std::make_tuple(i, j);
18     if (rcd.find(input) != rcd.end())
19         return rcd[input];
20
21     if (s1[i] == s2[j]) {
22         Output r =
23             lcm_helper(s1, s2, i - 1, j - 1, rcd) + 1;
24         rcd[input] = r;
```

```
25         return r;
26     }
27
28     Output r = std::max(
29         lcm_helper(s1, s2, i - 1, j, rcd),
30         lcm_helper(s1, s2, i, j - 1, rcd));
31     rcd[input] = r;
32     return r;
33 }
34
35 Output lcm(const std::string &s1,
36           const std::string &s2) {
37     Record rcd;
38     return lcm_helper(
39         s1,
40         s2,
41         s1.size() - 1,
42         s2.size() - 1,
43         rcd);
44 }
```

3.2 典型的动态规划问题

如果用数组的话，实际上效率会高很多：

```
1 using std::string;
2 using std::vector;
3
4 size_t better_lcm(const string &s1, const string &s2) {
5     size_t n1 = s1.size();
6     size_t n2 = s2.size();
7
8     vector<vector<size_t>> rec(n1 + 1, vector<size_t>(n2 + 1, 0));
9
10    for (size_t i = 1; i <= n1; i++) {
11        for (size_t j = 1; j <= n2; j++) {
12            if (s1[i - 1] == s2[j - 1]) {
13                rec[i][j] = rec[i - 1][j - 1] + 1;
14            } else {
15                rec[i][j] = std::max(rec[i][j - 1], rec[i - 1][j]);
16            }
17        }
18    }
19    return rec[n1][n2];
20 }
```

3.2 典型的动态规划问题

3.2.4 路径数问题

此问题选自 Advent of Code 2025 Day 11¹.

已知输入是一个单向图，每一行表示某个结点的后续连接节点列表。

```
1 svr: aaa bbb
2 aaa: fft
3 fft: ccc
4 bbb: tty
5 tty: ccc
6 ccc: ddd eee
7 ddd: hub
8 hub: fff
9 eee: dac
10 dac: fff
11 fff: ggg hhh
12 ggg: out
13 hhh: out
```

问：从 svr 节点出发，经过 fft 结点与 dac 节点后，到达 out 的路线，总共有多少种？

¹<https://adventofcode.com/2025/day/11>

3.2 典型的动态规划问题

从节点 i 到节点 j 的所有路线，相当于是节点 i 所有后续节点到结点 j 的路线的和。因为每个节点可能是多个节点的后续节点，因此需要记忆，避免重复计算。

3.2 典型的动态规划问题

3. 动态规划

```
1 #include <fstream>
2 #include <iostream>
3 #include <sstream>
4 #include <string>
5 #include <unordered_map>
6 #include <vector>
7
8 using namespace std;
9
10 using Node = string;
11 using Graph = unordered_map<Node, vector<Node>>;
12 using Count = unsigned long long;
13 using Record = unordered_map<Node, Count>;
14
15 void parse_line(const string &line, Graph &graph) {
16     istringstream in(line);
17     Node n1;
18     in >> n1;
19     n1 = n1.substr(0, 3);
20     vector<Node> outs;
21     Node tmp;
22     while (in >> tmp) {
23         outs.push_back(tmp);
24     }
25     graph[n1] = outs;
26 }
27
28 Count count_helper(const Graph &graph, const Node &from,
29                   const Node &to, Record &rec) {
30     if (from == to)
31         return 1;
32     if (rec.find(from) != rec.end())
33         return rec[from];
```

```
34     if (graph.find(from) == graph.end())
35         return 0;
36     auto nexts = graph.at(from);
37     Count sum = 0;
38     for (const auto &n : nexts) {
39         sum += count_helper(graph, n, to, rec);
40     }
41     rec[from] = sum;
42     return sum;
43 }
44
45 Count count(const Graph &graph, const Node &from, const Node &to) {
46     Record rec;
47     return count_helper(graph, from, to, rec);
48 }
49
50 int main(int argc, char **argv) {
51     ifstream file(argv[1]);
52     Graph graph;
53     string line;
54     while (getline(file, line)) {
55         parse_line(line, graph);
56     }
57     Count svr_fft = count(graph, "svr", "fft");
58     Count fft_dac = count(graph, "fft", "dac");
59     Count dac_out = count(graph, "dac", "out");
60     Count svr_dac = count(graph, "svr", "dac");
61     Count dac_fft = count(graph, "dac", "fft");
62     Count fft_out = count(graph, "fft", "out");
63     cout << svr_fft * fft_dac * dac_out +
64           svr_dac * dac_fft * fft_out
65           << "\n";
66 }
```

3.2 典型的动态规划问题

3.2.5 魔法石

此题选自 Advent of Code 2025 Day 11¹.

有一系列刻有数字的魔法石，当我们观察它们时，无事发生。只要没有观察它们（眨一下眼睛），它们就会发生变化，变化规律如下：

- 数字0会变为数字1
- 如果数字的长度是偶数，则左右分成两个数字，并去除数字的前置 0。比如，1001会被分为10和1
- 如果以上条件都不成立，则数字乘以 2024
- 无论如何变化，数字的顺序是为变的

问：眨了75次眼睛后，总共有多少块石头？

¹<https://adventofcode.com/2024/day/11>

3.2 典型的动态规划问题

问题的转换已经写明了，有不同的记忆方式。不过，与之前不一样的是，我们这次不再记忆参数的值，而是记忆对应数字的石头有多少块。

3.2 典型的动态规划问题

3. 动态规划

```
1 #include <fstream>
2 #include <iostream>
3 #include <string>
4 #include <unordered_map>
5 #include <vector>
6
7 using Stone = unsigned long long;
8 using Count = unsigned long long;
9 using Record = std::unordered_map<Stone, Count>;
10 using Stones = std::vector<Stone>;
11
12 Stones transform(Stone s) {
13     if (s == 0) return {1};
14
15     std::string str = std::to_string(s);
16     if (str.size() % 2 == 0) {
17         auto half = str.size() / 2;
18         auto s1 = str.substr(0, half);
19         auto s2 = str.substr(half);
20         return {(Stone)std::atoll(s1.c_str()),
21                 (Stone)std::atoll(s2.c_str())};
22     }
23
24     return {s * 2024};
25 }
26
27 Record blink(const Record &rec) {
28     Record new_stones;
```

```
29     for (auto &pair : rec) {
30         auto stone = pair.first;
31         auto count = pair.second;
32         auto generated = transform(stone);
33         for (auto ns : generated) new_stones[ns] += count;
34     }
35     return new_stones;
36 }
37
38 Count count(const Record &rec) {
39     Count s = 0;
40     for (auto &pair : rec) s += pair.second;
41     return s;
42 }
43
44 int main(int argc, char **argv) {
45     std::fstream file(argv[1]);
46     Record rec;
47     std::string s;
48     while (file >> s)
49         rec[(Stone)std::atoll(s.c_str())] += 1;
50     for (int i = 0; i < 75; i++) rec = blink(rec);
51     std::cout << count(rec) << "\n";
52 }
```

4. 总结

- 其本质上是一种结果记忆+数学归纳法

- 其本质上是一种结果记忆+数学归纳法
- 通过问题分解，将复杂问题分解为更加简单的问题

- 其本质上是一种结果记忆+数学归纳法
- 通过问题分解，将复杂问题分解为更加简单的问题
- 通过结果记忆，避免重复计算

- 其本质上是一种结果记忆+数学归纳法
- 通过问题分解，将复杂问题分解为更加简单的问题
- 通过结果记忆，避免重复计算
- 如果可以采用数组进行记忆，效率会更高

- 其本质上是一种结果记忆+数学归纳法
- 通过问题分解，将复杂问题分解为更加简单的问题
- 通过结果记忆，避免重复计算
- 如果可以采用数组进行记忆，效率会更高
- 不一定是记忆函数的参数，根据实际情况灵活变动

- 其本质上是一种结果记忆+数学归纳法
- 通过问题分解，将复杂问题分解为更加简单的问题
- 通过结果记忆，避免重复计算
- 如果可以采用数组进行记忆，效率会更高
- 不一定是记忆函数的参数，根据实际情况灵活变动
- 简单方法是采用 `unordered_map` 或者是 `map`

- 其本质上是一种结果记忆+数学归纳法
- 通过问题分解，将复杂问题分解为更加简单的问题
- 通过结果记忆，避免重复计算
- 如果可以采用数组进行记忆，效率会更高
- 不一定是记忆函数的参数，根据实际情况灵活变动
- 简单方法是采用 `unordered_map` 或者是 `map`
- C++中，复杂的数据类型往往不能保存到 `unordered_map` 中，除非手动实现 `hash` 函数。在一般的其他编程语言中没有这一问题（如 Python 中的 `dict`，Rust 中的 `HashMap` 等等）

- 其本质上是一种结果记忆+数学归纳法
- 通过问题分解，将复杂问题分解为更加简单的问题
- 通过结果记忆，避免重复计算
- 如果可以采用数组进行记忆，效率会更高
- 不一定是记忆函数的参数，根据实际情况灵活变动
- 简单方法是采用 `unordered_map` 或者是 `map`
- C++中，复杂的数据类型往往不能保存到 `unordered_map` 中，除非手动实现 `hash` 函数。在一般的其他编程语言中没有这一问题（如 Python 中的 `dict`，Rust 中的 `HashMap` 等等）
- C++由于历史包袱太重，导致语言比较难用。建议有兴趣的同学可以尝试一下其他现代语言。

5. 扩展：现代语言

- 现代语言实际上在语言表达能力、内在安全性上，都比 C++ 好用不少。
- 我们以 Rust 语言为例，看一下前面的两个问题如何表达。
- 最后我们会看看惰性求值的方式下的记忆策略（选择性学习，扩展内容，以 Haskell 语言为例）。

5.1 回看硬币找零

5. 扩展：现代语言

```
1 use std::collections::HashMap;
2
3 type Money = i32;
4 type Coins = [Money];
5 type Count = u64;
6 type Record<'a> =
7     HashMap<(&'a Coins, Money), Count>;
8
9 fn main() {
10     let coins =
11         [1, 2, 5, 10, 20, 50, 100, 200];
12     let target: Money = 200;
13     let mut memo: Record = HashMap::new();
14     let result =
15         solve(&coins, target, &mut memo);
16     println!("{}", result);
17 }
18
19 fn solve<'a>(  

```

```
20 coins: &'a Coins,
21 target: Money,
22 memo: &mut Record<'a>) -> u64 {
23     if let Some(&cached) =
24         memo.get(&(coins, target)) {
25         return cached;
26     }
27
28     let result = match (coins, target) {
29         (_, 0) => 1,
30         ([], _) => 0,
31         (_, t) if t < 0 => 0,
32         ([first, rest @ ..], t) =>
33             solve(coins, t - first, memo) +
34             solve(rest, t, memo),
35     };
36     memo.insert((coins, target), result);
37     result
38 }
```

5.2 魔法石问题偏函数式解法

5. 扩展：现代语言

```
1 use std::collections::HashMap;
2
3 type Stone = u64;
4 type Count = u64;
5 type Record = HashMap<Stone, Count>;
6
7 fn transform(s: Stone) -> Vec<Stone> {
8     if s == 0 {
9         return vec![1];
10    }
11    let digits =
12        (s as f64).log10().floor() as u32 + 1;
13    if digits.is_multiple_of(2) {
14        let divisor = 10u64.pow(digits / 2);
15        vec![s / divisor, s % divisor]
16    } else {
17        vec![s * 2024]
18    }
19 }
20
21 fn blink(rec: Record) -> Record {
22     let mut next_gen = Record::new();
23     for (stone, count) in rec {
24         for next_stone in transform(stone) {
25             *next_gen.entry(next_stone)
```

```
26                 .or_insert(0) += count;
27         }
28     }
29     next_gen
30 }
31
32 fn main() {
33     let input = std::env::args()
34         .nth(1)
35         .expect("Please provide an input file path");
36     let stones_text =
37         std::fs::read_to_string(input).unwrap();
38     let record = stones_text
39         .split_whitespace()
40         .filter_map(|s| s.parse::<Stone>().ok())
41         .fold(Record::new(), |mut acc, stone| {
42             *acc.entry(stone).or_insert(0) += 1;
43             acc
44         });
45     let record = (0..75).fold(
46         record, |rec, _| blink(rec));
47     println!("{}",
48         record.values().sum::<Count>());
49 }
```

5.3 惰性求值硬币问题（选）

此处是可选内容

还记得函数的描述么？把指令打包起来，暂不执行。

实际上，利用这一特性，我们可以构建一个称为 Thunk 的结构，它的内容包含了求值的指令，和求值的结果。在不需要它的值的时候，不求值。在第一次需要它的值的时候，执行指令并把计算到的结果保存好。之后，再需要它的值的时候，直接返回之前的计算结果。

听起来很美好，但是把程序的所有表达式变成 Thunk，有一个严苛到极致的需求：不允许变量的存在。我们必需确保每次求值的结果都是一样的。

Haskell 语言把惰性求值作为语言的默认行为。

5.3 惰性求值硬币问题（选）

在没有变量的前提下，如何实现记忆化？

一种典型的方法是利用惰性列表结构：

```
1 fibs :: [Integer]
2 fibs = 0 : 1 : zipWith (+) fibs (tail fibs)
3
4 main :: IO ()
5 main = print $ fibs !! 50 -- 打印第 50 个斐波那契数列
```

我们再来看看硬币找零问题：

5.3 惰性求值硬币问题 (选)

```
1 type Money = Int
2 type Coin = Money
3 type Count = Integer
4 solve :: Money -> [Coin] -> Count
5 solve target coins =
6   foldl' step initSolution coins !! target
7   where
8     initSolution = 1 : repeat 0
9     step s coin =
10       let s' = take coin s ++ zipWith (+) (drop coin s) s'
11       in s'
12 main :: IO ()
13 main = print $ solve 200 [1, 2, 5, 10, 20, 50, 100, 200]
```